

Operating Systems

Sem 3

Watermarked study resource

Generated from local SRM dataset

===== SOURCE_FILE: sem 3/Operating Systems/Notes/Notes From NCR Campus By BTECH CRUX.url

=====

CATEGORY: Notes

STATUS: url_reference_only

URL: https://drive.google.com/drive/folders/1Qt82PVY7dtlQZVS7ajLqAzaTZbYCQCq9?usp=drive_link

===== SOURCE_FILE: sem 3/Operating Systems/Notes/Unit 1.pdf =====

CATEGORY: Notes

STATUS: ok

[Page 1]

21CSC202J Operating Systems

UNIT I

- Outline the structure of OS and basic architectural Component is involved in OS design

Course

Learning

Rationale

- Use the appropriate Concepts of Operating System for Resource Utilization

Course

Learning

Outcomes

Learning Resources

1.Abraham Silberschatz, Peter Baer Galvin, Greg Gagne, Operating systems, 9th ed., John Wiley & Sons, 2013

[Page 2]

Introduction

- What Operating Systems Do
- Computer-System Organization
- Computer-System Architecture
- Operating-System Structure
- Operating-System Operations

- Process Management
 - Memory Management
 - Storage Management
 - Protection and Security
 - Kernel Data Structures
 - Computing Environments
 - Open-Source Operating Systems
- 8/3/2023 21CSC202J Operating Systems
- UNIT 1 2

[Page 3]

Objectives

- To describe the basic organization of computer systems
- To provide a grand tour of the major components of operating systems
- To give an overview of the many types of computing environments
- To explore several open-source operating systems

8/3/2023 21CSC202J Operating Systems

UNIT 1 3

[Page 4]

What is an Operating System?

- A program that acts as an intermediary between a user of a computer and the computer hardware
- Operating system goals:
 - Execute user programs and make solving user problems easier
 - Make the computer system convenient to use
 - Use the computer hardware in an efficient manner

8/3/2023 21CSC202J Operating Systems

UNIT 1 4

[Page 5]

Computer System Structure

- Computer system can be divided into four components:
 - Hardware – provides basic computing resources
- CPU, memory, I/O devices
- Operating system
- Controls and coordinates use of hardware among various applications and users
- Application programs – define the ways in which the system resources are used to solve the computing problems of the users
- Word processors, compilers, web browsers, database systems, video games

– Users

- People, machines, other computers

8/3/2023 21CSC202J Operating Systems

UNIT 1 5

[Page 6]

Four Components of a Computer System

8/3/2023 21CSC202J Operating Systems

UNIT 1 6

[Page 7]

What Operating Systems Do

- Depends on the point of view
- Users want convenience, ease of use and good performance
 - Don't care about resource utilization
- But shared computer such as mainframe or minicomputer must keep all users happy
- Users of dedicated systems such as workstations have dedicated resources but frequently use shared resources from servers
- Handheld computers are resource poor, optimized for usability and battery life
- Some computers have little or no user interface, such as embedded computers in devices and automobiles

8/3/2023 21CSC202J Operating Systems

UNIT 1 7

[Page 8]

Operating System Definition

- OS is a resource allocator
 - Manages all resources
 - Decides between conflicting requests for efficient and fair resource use
- OS is a control program
 - Controls execution of programs to prevent errors and improper use of the computer

8/3/2023 21CSC202J Operating Systems

UNIT 1 8

[Page 9]

Operating System Definition (Cont.)

- No universally accepted definition
- “Everything a vendor ships when you order an operating system” is a good approximation
 - But varies wildly
- “The one program running at all times on the computer” is

the kernel.

- Everything else is either
 - a system program (ships with the operating system) , or
 - an application program.

8/3/2023 21CSC202J Operating Systems

UNIT 1 9

[Page 10]

Computer Startup

- bootstrap program is loaded at power-up or reboot
 - Typically stored in ROM or EPROM, generally known as firmware
 - Initializes all aspects of system
 - Loads operating system kernel and starts execution

8/3/2023 21CSC202J Operating Systems

UNIT 1 10

[Page 11]

Computer System Organization

- Computer-system operation
 - One or more CPUs, device controllers connect through common bus providing access to shared memory
 - Concurrent execution of CPUs and devices competing for memory cycles

8/3/2023 21CSC202J Operating Systems

UNIT 1 11

[Page 12]

Computer-System Operation

- I/O devices and the CPU can execute concurrently
- Each device controller is in charge of a particular device type
- Each device controller has a local buffer
- CPU moves data from/to main memory to/from local buffers
- I/O is from the device to local buffer of controller
- Device controller informs CPU that it has finished its operation by causing an interrupt

8/3/2023 21CSC202J Operating Systems

UNIT 1 12

[Page 13]

Common Functions of Interrupts

- Interrupt transfers control to the interrupt service routine generally, through the interrupt vector, which contains the addresses of all the service routines
- Interrupt architecture must save the address of the interrupted instruction
- A trap or exception is a software-generated interrupt caused either by an error or a user request
- An operating system is interrupt driven

8/3/2023 21CSC202J Operating Systems

UNIT 1 13

[Page 14]

Interrupt Handling

- The operating system preserves the state of the CPU by storing registers and the program counter
- Determines which type of interrupt has occurred:
 - polling
 - vectored interrupt system
- Separate segments of code determine what action should be taken for each type of interrupt

8/3/2023 21CSC202J Operating Systems

UNIT 1 14

[Page 15]

Interrupt Timeline

8/3/2023 21CSC202J Operating Systems

UNIT 1 15

[Page 16]

I/O Structure

- After I/O starts, control returns to user program only upon I/O completion
 - Wait instruction idles the CPU until the next interrupt
 - Wait loop (contention for memory access)
 - At most one I/O request is outstanding at a time, no simultaneous I/O processing
- After I/O starts, control returns to user program without waiting for I/O completion
 - System call - request to the OS to allow user to wait for I/O completion
 - Device-status table contains entry for each I/O device indicating its type, address, and state
 - OS indexes into I/O device table to determine device status and to modify table entry to include interrupt

8/3/2023 21CSC202J Operating Systems

[Page 17]

Storage Definitions and Notation Review

The basic unit of computer storage is the bit. A bit can contain one of two values, 0 and 1. All other storage in a computer is based on collections of bits. Given enough bits, it is amazing how many things a computer can represent: numbers, letters, images, movies, sounds, documents, and programs, to name a few. A byte is 8 bits, and on most computers it is the smallest convenient chunk of storage. For example, most computers don't have an instruction to move a bit but do have one to move a byte. A less common term is word, which is a given computer architecture's native unit of data. A word is made up of one or more bytes. For example, a computer that has 64-bit registers and 64-bit memory addressing typically has 64-bit (8-byte) words. A computer executes many operations in its native word size rather than a byte at a time. Computer storage, along with most computer throughput, is generally measured and manipulated in bytes and collections of bytes.

A kilobyte, or KB, is 1,024 bytes

a megabyte, or MB, is 1,024² bytes

a gigabyte, or GB, is 1,024³ bytes

a terabyte, or TB, is 1,024⁴ bytes

a petabyte, or PB, is 1,024⁵ bytes

Computer manufacturers often round off these numbers and say that a megabyte is 1 million bytes and a gigabyte is 1 billion bytes. Networking measurements are an exception to this general rule; they are given in bits (because networks move data a bit at a time).

8/3/2023 21CSC202J Operating Systems

[Page 18]

Storage Structure

- Main memory – only large storage media that the CPU can access directly
 - Random access
 - Typically volatile
- Secondary storage – extension of main memory that provides large nonvolatile storage capacity
 - Hard disks – rigid metal or glass platters covered with magnetic recording material
 - Disk surface is logically divided into tracks, which are subdivided into sectors
 - The disk controller determines the logical interaction between the device and the computer
 - Solid-state disks – faster than hard disks, nonvolatile
 - Various technologies
 - Becoming more popular

[Page 19]

Storage Hierarchy

- Storage systems organized in hierarchy
 - Speed
 - Cost
 - Volatility
- Caching - copying information into faster storage system; main memory can be viewed as a cache for secondary storage
- Device Driver for each device controller to manage I/O
 - Provides uniform interface between controller and kernel

[Page 20]

Storage-Device Hierarchy

[Page 21]

Caching

- Important principle, performed at many levels in a computer (in hardware, operating system, software)
- Information in use copied from slower to faster storage temporarily
- Faster storage (cache) checked first to determine if information is there
 - If it is, information used directly from the cache (fast)
 - If not, data copied to cache and used there
- Cache smaller than storage being cached
 - Cache management important design problem
 - Cache size and replacement policy

[Page 22]

Direct Memory Access Structure

- Used for high-speed I/O devices able to transmit information at close to memory speeds
- Device controller transfers blocks of data from buffer storage directly to main memory without CPU intervention
- Only one interrupt is generated per block, rather than the

one interrupt per byte

8/3/2023 21CSC202J Operating Systems

UNIT 1 22

[Page 23]

How a Modern Computer Works

A von Neumann architecture

8/3/2023 21CSC202J Operating Systems

UNIT 1 23

[Page 24]

Computer-System Architecture

- Most systems use a single general-purpose processor
- Most systems have special-purpose processors as well
- Multiprocessors systems growing in use and importance
- Also known as parallel systems, tightly-coupled systems
- Advantages include:
 1. Increased throughput
 2. Economy of scale
 3. Increased reliability - graceful degradation or fault tolerance
- Two types:
 1. Asymmetric Multiprocessing - each processor is assigned a specific task.
 2. Symmetric Multiprocessing - each processor performs all tasks

8/3/2023 21CSC202J Operating Systems

UNIT 1 24

[Page 25]

Symmetric Multiprocessing Architecture

8/3/2023 21CSC202J Operating Systems

UNIT 1 25

[Page 26]

A Dual-Core Design

- Multi-chip and multicore
- Systems containing all chips
- Chassis containing multiple separate systems

8/3/2023 21CSC202J Operating Systems

UNIT 1 26

[Page 27]

Clustered Systems

- Like multiprocessor systems, but multiple systems working together

- Usually sharing storage via a storage-area network (SAN)
- Provides a high-availability service which survives failures
 - Asymmetric clustering has one machine in hot-standby mode
 - Symmetric clustering has multiple nodes running applications, monitoring each other
- Some clusters are for high-performance computing (HPC)
 - Applications must be written to use parallelization
- Some have distributed lock manager (DLM) to avoid conflicting operations

8/3/2023 21CSC202J Operating Systems

UNIT 1 27

[Page 28]

Clustered Systems

8/3/2023 21CSC202J Operating Systems

UNIT 1 28

[Page 29]

Operating System Structure

- Multiprogramming (Batch system) needed for efficiency
 - Single user cannot keep CPU and I/O devices busy at all times
 - Multiprogramming organizes jobs (code and data) so CPU always has one to execute
 - A subset of total jobs in system is kept in memory
 - One job selected and run via job scheduling
 - When it has to wait (for I/O for example), OS switches to another job
- Timesharing (multitasking) is logical extension in which CPU switches jobs so frequently that users can interact with each job while it is running, creating interactive computing
 - Response time should be < 1 second
 - Each user has at least one program executing in memory → process
 - If several jobs ready to run at the same time → CPU scheduling
 - If processes don't fit in memory, swapping moves them in and out to run
 - Virtual memory allows execution of processes not completely in memory

8/3/2023 21CSC202J Operating Systems

UNIT 1 29

[Page 30]

Memory Layout for Multiprogrammed System

8/3/2023 21CSC202J Operating Systems

UNIT 1 30

[Page 31]

Operating-System Operations

- Interrupt driven (hardware and software)
 - Hardware interrupt by one of the devices
 - Software interrupt (exception or trap):
- Software error (e.g., division by zero)
- Request for operating system service
- Other process problems include infinite loop, processes modifying each other or the operating system

8/3/2023 21CSC202J Operating Systems

UNIT 1 31

[Page 32]

Operating-System Operations (cont.)

- Dual-mode operation allows OS to protect itself and other system components
 - User mode and kernel mode
 - Mode bit provided by hardware
- Provides ability to distinguish when system is running user code or kernel code
- Some instructions designated as privileged, only executable in kernel mode
- System call changes mode to kernel, return from call resets it to user
- Increasingly CPUs support multi-mode operations
 - i.e. virtual machine manager (VMM) mode for guest VMs

8/3/2023 21CSC202J Operating Systems

UNIT 1 32

[Page 33]

Transition from User to Kernel Mode

- Timer to prevent infinite loop / process hogging resources
 - Timer is set to interrupt the computer after some time period
 - Keep a counter that is decremented by the physical clock.
 - Operating system set the counter (privileged instruction)
 - When counter zero generate an interrupt
 - Set up before scheduling process to regain control or terminate program that exceeds allotted time

8/3/2023 21CSC202J Operating Systems

UNIT 1 33

[Page 34]

Process Management

- A process is a program in execution. It is a unit of work within the system. Program is a passive entity, process is an active entity.
- Process needs resources to accomplish its task
 - CPU, memory, I/O, files
 - Initialization data
- Process termination requires reclaim of any reusable resources
- Single-threaded process has one program counter specifying location of next instruction to execute
 - Process executes instructions sequentially, one at a time, until completion
- Multi-threaded process has one program counter per thread
- Typically system has many processes, some user, some operating system running concurrently on one or more CPUs
 - Concurrency by multiplexing the CPUs among the processes / threads

8/3/2023 21CSC202J Operating Systems

UNIT 1 34

[Page 35]

Process Management Activities

- Creating and deleting both user and system processes
 - Suspending and resuming processes
 - Providing mechanisms for process synchronization
 - Providing mechanisms for process communication
 - Providing mechanisms for deadlock handling
- The operating system is responsible for the following activities in connection with process management:

8/3/2023 21CSC202J Operating Systems

UNIT 1 35

[Page 36]

Memory Management

- To execute a program all (or part) of the instructions must be in memory
- All (or part) of the data that is needed by the program must be in memory.
- Memory management determines what is in memory and when
 - Optimizing CPU utilization and computer response to users
- Memory management activities
 - Keeping track of which parts of memory are currently being used and by whom
 - Deciding which processes (or parts thereof) and data to move into and out of memory
 - Allocating and deallocating memory space as needed

[Page 37]

Storage Management

- OS provides uniform, logical view of information storage
 - Abstracts physical properties to logical storage unit - file
 - Each medium is controlled by device (i.e., disk drive, tape drive)
- Varying properties include access speed, capacity, data-transfer rate, access method (sequential or random)
- File-System management
 - Files usually organized into directories
 - Access control on most systems to determine who can access what
- OS activities include
 - Creating and deleting files and directories
 - Primitives to manipulate files and directories
 - Mapping files onto secondary storage
 - Backup files onto stable (non-volatile) storage media

[Page 38]

Mass-Storage Management

- Usually disks used to store data that does not fit in main memory or data that must be kept for a “long” period of time
- Proper management is of central importance
- Entire speed of computer operation hinges on disk subsystem and its algorithms
- OS activities
 - Free-space management
 - Storage allocation
 - Disk scheduling
- Some storage need not be fast
 - Tertiary storage includes optical storage, magnetic tape
 - Still must be managed - by OS or applications
 - Varies between WORM (write-once, read-many-times) and RW (read-write)

[Page 39]

Performance of Various Levels of Storage

Movement between levels of storage hierarchy can be explicit or implicit

[Page 40]

Migration of data "A" from Disk to Register

- Multitasking environments must be careful to use most recent value, no matter where it is stored in the storage hierarchy
- Multiprocessor environment must provide cache coherency in hardware such that all CPUs have the most recent value in their cache
- Distributed environment situation even more complex
 - Several copies of a datum can exist

8/3/2023 21CSC202J Operating Systems

UNIT 1 40

[Page 41]

I/O Subsystem

- One purpose of OS is to hide peculiarities of hardware devices from the user
- I/O subsystem responsible for
 - Memory management of I/O including buffering (storing data temporarily while it is being transferred), caching (storing parts of data in faster storage for performance), spooling (the overlapping of output of one job with input of other jobs)
 - General device-driver interface
 - Drivers for specific hardware devices

8/3/2023 21CSC202J Operating Systems

UNIT 1 41

[Page 42]

Protection and Security

- Protection - any mechanism for controlling access of processes or users to resources defined by the OS
- Security - defense of the system against internal and external attacks
 - Huge range, including denial-of-service, worms, viruses, identity theft, theft of service
- Systems generally first distinguish among users, to determine who can do what
 - User identities (user IDs, security IDs) include name and associated number, one per user
 - User ID then associated with all files, processes of that user to determine access control
 - Group identifier (group ID) allows set of users to be defined and controls managed, then also associated with each process, file
 - Privilege escalation allows user to change to effective ID with more rights

8/3/2023 21CSC202J Operating Systems

UNIT 1 42

[Page 43]

Kernel Data Structures

Many similar to standard programming data structures

Singly linked list

Doubly linked list

Circular linked list

8/3/2023 21CSC202J Operating Systems

UNIT 1 43

[Page 44]

Kernel Data Structures

- Binary search tree

left $\leq$ right

- Search performance is $O(n)$
- Balanced binary search tree is $O(\lg n)$

8/3/2023 21CSC202J Operating Systems

UNIT 1 44

[Page 45]

Kernel Data Structures

- Hash function can create a hash map
- Bitmap - string of n binary digits representing the status of n items
- Linux data structures defined in

include files `<linux/list.h>`, `<linux/kfifo.h>`,
`<linux/rbtree.h>`

8/3/2023 21CSC202J Operating Systems

UNIT 1 45

[Page 46]

Computing Environments - Traditional

- Stand-alone general purpose machines
- But blurred as most systems interconnect with others (i.e., the Internet)
- Portals provide web access to internal systems
- Network computers (thin clients) are like Web terminals
- Mobile computers interconnect via wireless networks
- Networking becoming ubiquitous - even home systems use firewalls to protect home computers from Internet attacks

8/3/2023 21CSC202J Operating Systems

UNIT 1 46

[Page 47]

Computing Environments - Mobile

- Handheld smartphones, tablets, etc

- What is the functional difference between them and a “traditional” laptop?
- Extra feature – more OS features (GPS, gyroscope)
- Allows new types of apps like augmented reality
- Use IEEE 802.11 wireless, or cellular data networks for connectivity
- Leaders are Apple iOS and Google Android

8/3/2023 21CSC202J Operating Systems

UNIT 1 47

[Page 48]

Computing Environments – Distributed

- Distributed computing
 - Collection of separate, possibly heterogeneous, systems networked together
- Network is a communications path, TCP/IP most common
 - Local Area Network (LAN)
 - Wide Area Network (WAN)
 - Metropolitan Area Network (MAN)
 - Personal Area Network (PAN)
 - Network Operating System provides features between systems across network
- Communication scheme allows systems to exchange messages
- Illusion of a single system

8/3/2023 21CSC202J Operating Systems

UNIT 1 48

[Page 49]

Computing Environments – Client-Server

Client-Server Computing

Dumb terminals supplanted by smart PCs

Many systems now servers, responding to requests generated by clients

☐ Compute-server system provides an interface to client to request services (i.e., database)

☐ File-server system provides interface for clients to store and retrieve files

8/3/2023 21CSC202J Operating Systems

UNIT 1 49

[Page 50]

Computing Environments – Peer-to-Peer

- Another model of distributed system
- P2P does not distinguish clients and servers
 - Instead all nodes are considered peers
 - May each act as client, server or both

- Node must join P2P network
- Registers its service with central lookup service on network, or
- Broadcast request for service and respond to requests for service via discovery protocol
- Examples include Napster and Gnutella, Voice over IP (VoIP) such as Skype

8/3/2023 21CSC202J Operating Systems

UNIT 1 50

[Page 51]

Computing Environments - Virtualization

- Allows operating systems to run applications within other OSes
- Vast and growing industry
- Emulation used when source CPU type different from target type (i.e. PowerPC to Intel x86)
- Generally slowest method
- When computer language not compiled to native code - Interpretation
- Virtualization - OS natively compiled for CPU, running guest OSes also natively compiled
- Consider VMware running WinXP guests, each running applications, all on native WinXP host OS
- VMM (virtual machine Manager) provides virtualization services

8/3/2023 21CSC202J Operating Systems

UNIT 1 51

[Page 52]

Computing Environments - Virtualization

- Use cases involve laptops and desktops running multiple OSes for exploration or compatibility
- Apple laptop running Mac OS X host, Windows as a guest
- Developing apps for multiple OSes without having multiple systems
- QA testing applications without having multiple systems
- Executing and managing compute environments within data centers
- VMM can run natively, in which case they are also the host
- There is no general purpose host then (VMware ESX and Citrix XenServer)

8/3/2023 21CSC202J Operating Systems

UNIT 1 52

[Page 53]

Computing Environments -

Virtualization

8/3/2023 21CSC202J Operating Systems

UNIT 1 53

[Page 54]

Computing Environments -

Cloud Computing

- Delivers computing, storage, even apps as a service across a network
- Logical extension of virtualization because it uses virtualization as the base for its functionality.
- Amazon EC2 has thousands of servers, millions of virtual machines, petabytes of storage available across the Internet, pay based on usage
- Many types
 - Public cloud - available via Internet to anyone willing to pay
 - Private cloud - run by a company for the company's own use
 - Hybrid cloud - includes both public and private cloud components
 - Software as a Service (SaaS) - one or more applications available via the Internet (i.e., word processor)
 - Platform as a Service (PaaS) - software stack ready for application use via the Internet (i.e., a database server)
 - Infrastructure as a Service (IaaS) - servers or storage available over Internet (i.e., storage available for backup use)

8/3/2023 21CSC202J Operating Systems

UNIT 1 54

[Page 55]

Computing Environments -

Cloud Computing

- Cloud computing environments composed of traditional OSes, plus VMs, plus cloud management tools
- Internet connectivity requires security like firewalls
- Load balancers spread traffic across multiple applications

8/3/2023 21CSC202J Operating Systems

UNIT 1 55

[Page 56]

Computing Environments -

Real-Time Embedded Systems

- Real-time embedded systems most prevalent form of computers
- Vary considerable, special purpose, limited purpose OS, real-time OS

- Use expanding
 - Many other special computing environments as well
- Some have Oses, some perform tasks without an OS
 - Real-time OS has well-defined fixed time constraints
- Processing must be done within constraint
- Correct operation only if constraints met

8/3/2023 21CSC202J Operating Systems

UNIT 1 56

[Page 57]

Open-Source Operating Systems

- Operating systems made available in source-code format rather than just binary closed-source
- Counter to the copy protection and Digital Rights Management (DRM) movement
- Started by Free Software Foundation (FSF), which has “copyleft” GNU Public License (GPL)
- Examples include GNU/Linux and BSD UNIX (including core of Mac OS X), and many more
- Can use VMM like VMware Player (Free on Windows), Virtualbox (open source and free on many platforms - <http://www.virtualbox.com>)
- Use to run guest operating systems for exploration

8/3/2023 21CSC202J Operating Systems

UNIT 1 57

[Page 58]

Operating-System Structures

- Operating System Services
- User Operating System Interface
- System Calls
- Types of System Calls
- System Programs
- Operating System Design and Implementation
- Operating System Structure
- Operating System Debugging
- Operating System Generation
- System Boot

8/3/2023 21CSC202J Operating Systems

UNIT 1 58

[Page 59]

Objectives

- To describe the services an operating system provides to users, processes, and other systems

- To discuss the various ways of structuring an operating system
- To explain how operating systems are installed and customized and how they boot

8/3/2023 21CSC202J Operating Systems

UNIT 1 59

[Page 60]

Operating System Services

- Operating systems provide an environment for execution of programs and services to programs and users
- One set of operating-system services provides functions that are helpful to the user:
 - User interface - Almost all operating systems have a user interface (UI).
 - Varies between Command-Line (CLI), Graphics User Interface (GUI), Batch
 - Program execution - The system must be able to load a program into memory and to run that program, end execution, either normally or abnormally (indicating error)
 - I/O operations - A running program may require I/O, which may involve a file or an I/O device

8/3/2023 21CSC202J Operating Systems

UNIT 1 60

[Page 61]

Operating System Services (Cont.)

- One set of operating-system services provides functions that are helpful to the user (Cont.):
 - File-system manipulation - The file system is of particular interest. Programs need to read and write files and directories, create and delete them, search them, list file Information, permission management.
 - Communications - Processes may exchange information, on the same computer or between computers over a network
 - Communications may be via shared memory or through message passing (packets moved by the OS)
 - Error detection - OS needs to be constantly aware of possible errors
 - May occur in the CPU and memory hardware, in I/O devices, in user program
 - For each type of error, OS should take the appropriate action to ensure correct and consistent computing
 - Debugging facilities can greatly enhance the user's and programmer's abilities to efficiently use the system

8/3/2023 21CSC202J Operating Systems

UNIT 1 61

[Page 62]

Operating System Services (Cont.)

- Another set of OS functions exists for ensuring the efficient operation of the system itself via resource sharing
 - Resource allocation - When multiple users or multiple jobs running concurrently, resources must be allocated to each of them
- Many types of resources - CPU cycles, main memory, file storage, I/O devices.
 - Accounting - To keep track of which users use how much and what kinds of computer resources
 - Protection and security - The owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other
- Protection involves ensuring that all access to system resources is controlled
- Security of the system from outsiders requires user authentication, extends to defending external I/O devices from invalid access attempts

8/3/2023 21CSC202J Operating Systems

UNIT 1 62

[Page 63]

A View of Operating System Services

8/3/2023 21CSC202J Operating Systems

UNIT 1 63

[Page 64]

User Operating System Interface - CLI

CLI or command interpreter allows direct command entry

- Sometimes implemented in kernel, sometimes by systems program
- Sometimes multiple flavors implemented - shells
- Primarily fetches a command from user and executes it
- Sometimes commands built-in, sometimes just names of programs
- If the latter, adding new features doesn't require shell modification

8/3/2023 21CSC202J Operating Systems

UNIT 1 64

[Page 65]

Bourne Shell Command Interpreter

8/3/2023 21CSC202J Operating Systems

UNIT 1 65

[Page 66]

User Operating System Interface - GUI

- User-friendly desktop metaphor interface
 - Usually mouse, keyboard, and monitor
 - Icons represent files, programs, actions, etc
 - Various mouse buttons over objects in the interface cause various actions (provide information, options, execute function, open directory (known as a folder)
 - Invented at Xerox PARC
- Many systems now include both CLI and GUI interfaces
 - Microsoft Windows is GUI with CLI “command” shell
 - Apple Mac OS X is “Aqua”GUI interface with UNIX kernel underneath and shells available
 - Unix and Linux have CLI with optional GUI interfaces (CDE, KDE, GNOME)

8/3/2023 21CSC202J Operating Systems

UNIT 1 66

[Page 67]

Touchscreen Interfaces

n Touchscreen devices

require new

interfaces

l Mouse not possible or not desired

l Actions and selection based on gestures

l Virtual keyboard for text entry

l V oice commands.

8/3/2023 21CSC202J Operating Systems

UNIT 1 67

[Page 68]

The Mac OS X GUI

8/3/2023 21CSC202J Operating Systems

UNIT 1 68

[Page 69]

System Calls

- Programming interface to the services provided by the OS
- Typically written in a high-level language (C or C++)
- Mostly accessed by programs via a high-level Application Programming Interface (API) rather than direct system call use
- Three most common APIs are Win32 API for Windows, POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X), and Java API for the Java virtual machine (JVM)

Note that the system-call names used throughout this text are generic

8/3/2023 21CSC202J Operating Systems

UNIT 1 69

[Page 70]

Example of System Calls

- System call sequence to copy the contents of one file to another file

8/3/2023 21CSC202J Operating Systems

UNIT 1 70

[Page 71]

Example of Standard API

8/3/2023 21CSC202J Operating Systems

UNIT 1 71

[Page 72]

System Call Implementation

- Typically, a number associated with each system call
 - System-call interface maintains a table indexed according to these numbers
- The system call interface invokes the intended system call in OS kernel and returns status of the system call and any return values
- The caller need know nothing about how the system call is implemented
 - Just needs to obey API and understand what OS will do as a result call
 - Most details of OS interface hidden from programmer by API
- Managed by run-time support library (set of functions built into libraries included with compiler)

8/3/2023 21CSC202J Operating Systems

UNIT 1 72

[Page 73]

API - System Call - OS Relationship

8/3/2023 21CSC202J Operating Systems

UNIT 1 73

[Page 74]

System Call Parameter Passing

- Often, more information is required than simply identity of desired system call
 - Exact type and amount of information vary according to OS and call

- Three general methods used to pass parameters to the OS
 - Simplest: pass the parameters in registers
- In some cases, may be more parameters than registers
 - Parameters stored in a block, or table, in memory, and address of block passed as a parameter in a register
- This approach taken by Linux and Solaris
 - Parameters placed, or pushed, onto the stack by the program and popped off the stack by the operating system
 - Block and stack methods do not limit the number or length of parameters being passed

8/3/2023 21CSC202J Operating Systems

UNIT 1 74

[Page 75]

Parameter Passing via Table

8/3/2023 21CSC202J Operating Systems

UNIT 1 75

[Page 76]

Types of System Calls

- Process control
 - create process, terminate process
 - end, abort
 - load, execute
 - get process attributes, set process attributes
 - wait for time
 - wait event, signal event
 - allocate and free memory
 - Dump memory if error
 - Debugger for determining bugs, single step execution
 - Locks for managing access to shared data between processes

8/3/2023 21CSC202J Operating Systems

UNIT 1 76

[Page 77]

Types of System Calls

- File management
 - create file, delete file
 - open, close file
 - read, write, reposition
 - get and set file attributes
- Device management
 - request device, release device
 - read, write, reposition
 - get device attributes, set device attributes

- logically attach or detach devices

8/3/2023 21CSC202J Operating Systems

UNIT 1 77

[Page 78]

Types of System Calls (Cont.)

- Information maintenance

- get time or date, set time or date

- get system data, set system data

- get and set process, file, or device attributes

- Communications

- create, delete communication connection

- send, receive messages if message passing model to host name
or process name

- From client to server

- Shared-memory model create and gain access to memory
regions

- transfer status information

- attach and detach remote devices

8/3/2023 21CSC202J Operating Systems

UNIT 1 78

[Page 79]

Types of System Calls (Cont.)

- Protection

- Control access to resources

- Get and set permissions

- Allow and deny user access

8/3/2023 21CSC202J Operating Systems

UNIT 1 79

[Page 80]

Examples of Windows and Unix System Calls

8/3/2023 21CSC202J Operating Systems

UNIT 1 80

[Page 81]

Standard C Library Example

- C program invoking printf() library call,
which calls write() system call

8/3/2023 21CSC202J Operating Systems

UNIT 1 81

[Page 82]

Example: MS-DOS

- Single-tasking
- Shell invoked when system booted
- Simple method to run program
 - No process created
- Single memory space
- Loads program into memory, overwriting all but the kernel
- Program exit -> shell reloaded

At system startup running a program
 8/3/2023 21CSC202J Operating Systems
 UNIT 1 82

[Page 83]

Example: FreeBSD

- Unix variant
- Multitasking
- User login -> invoke user's choice of shell
- Shell executes fork() system call to create process
 - Executes exec() to load program into process
 - Shell waits for process to terminate or continues with user commands
- Process exits with:
 - code = 0 - no error
 - code > 0 - error code

8/3/2023 21CSC202J Operating Systems
 UNIT 1 83

[Page 84]

System Programs

- System programs provide a convenient environment for program development and execution. They can be divided into:
 - File manipulation
 - Status information sometimes stored in a File modification
 - Programming language support
 - Program loading and execution
 - Communications
 - Background services
 - Application programs

- Most users' view of the operation system is defined by system programs, not the actual system calls

8/3/2023 21CSC202J Operating Systems

UNIT 1 84

[Page 85]

System Programs

- Provide a convenient environment for program development and execution
 - Some of them are simply user interfaces to system calls; others are considerably more complex
- File management - Create, delete, copy, rename, print, dump, list, and generally manipulate files and directories
- Status information
 - Some ask the system for info - date, time, amount of available memory, disk space, number of users
 - Others provide detailed performance, logging, and debugging information
 - Typically, these programs format and print the output to the terminal or other output devices
 - Some systems implement a registry - used to store and retrieve configuration information

8/3/2023 21CSC202J Operating Systems

UNIT 1 85

[Page 86]

System Programs (Cont.)

- File modification
 - Text editors to create and modify files
 - Special commands to search contents of files or perform transformations of the text
- Programming-language support - Compilers, assemblers, debuggers and interpreters sometimes provided
- Program loading and execution- Absolute loaders, relocatable loaders, linkage editors, and overlay-loaders, debugging systems for higher-level and machine language
- Communications - Provide the mechanism for creating virtual connections among processes, users, and computer systems
 - Allow users to send messages to one another's screens, browse web pages, send electronic-mail messages, log in remotely, transfer files from one machine to another

8/3/2023 21CSC202J Operating Systems

UNIT 1 86

[Page 87]

System Programs (Cont.)

- Background Services
 - Launch at boot time
- Some for system startup, then terminate
- Some from system boot to shutdown
 - Provide facilities like disk checking, process scheduling, error logging, printing
 - Run in user context not kernel context
 - Known as services, subsystems, daemons
- Application programs
 - Don't pertain to system
 - Run by users
 - Not typically considered part of OS
 - Launched by command line, mouse click, finger poke

8/3/2023 21CSC202J Operating Systems

UNIT 1 87

[Page 88]

Operating System Design and Implementation

- Design and Implementation of OS not "solvable", but some approaches have proven successful
- Internal structure of different Operating Systems can vary widely
- Start the design by defining goals and specifications
- Affected by choice of hardware, type of system
- User goals and System goals
 - User goals - operating system should be convenient to use, easy to learn, reliable, safe, and fast
 - System goals - operating system should be easy to design, implement, and maintain, as well as flexible, reliable, error-free, and efficient

8/3/2023 21CSC202J Operating Systems

UNIT 1 88

[Page 89]

Operating System Design and Implementation (Cont.)

- Important principle to separate

Policy: What will be done?

Mechanism: How to do it?

- Mechanisms determine how to do something, policies decide what will be done
- The separation of policy from mechanism is a very important principle, it allows maximum flexibility if policy decisions are to be changed later (example - timer)

- Specifying and designing an OS is highly creative task of software engineering

8/3/2023 21CSC202J Operating Systems

UNIT 1 89

[Page 90]

Implementation

- Much variation
 - Early Oses in assembly language
 - Then system programming languages like Algol, PL/1
 - Now C, C++
- Actually usually a mix of languages
 - Lowest levels in assembly
 - Main body in C
 - Systems programs in C, C++, scripting languages like PERL, Python, shell scripts
- More high-level language easier to port to other hardware
 - But slower
- Emulation can allow an OS to run on non-native hardware

8/3/2023 21CSC202J Operating Systems

UNIT 1 90

[Page 91]

Operating System Structure

- General-purpose OS is very large program
- Various ways to structure ones
 - Simple structure - MS-DOS
 - More complex -- UNIX
 - Layered - an abstraction
 - Microkernel -Mach

8/3/2023 21CSC202J Operating Systems

UNIT 1 91

[Page 92]

Simple Structure -- MS-DOS

- MS-DOS - written to provide the most functionality in the least space
 - Not divided into modules
 - Although MS-DOS has some structure, its interfaces and levels of functionality are not

well separated

8/3/2023 21CSC202J Operating Systems

UNIT 1 92

[Page 93]

Non Simple Structure -- UNIX

UNIX - limited by hardware functionality, the original UNIX operating system had limited structuring. The UNIX OS consists of two separable parts

- Systems programs
- The kernel
 - Consists of everything below the system-call interface and above the physical hardware
 - Provides the file system, CPU scheduling, memory management, and other operating-system functions; a large number of functions for one level

8/3/2023 21CSC202J Operating Systems

UNIT 1 93

[Page 94]

Traditional UNIX System Structure

Beyond simple but not fully layered

8/3/2023 21CSC202J Operating Systems

UNIT 1 94

[Page 95]

Layered Approach

- The operating system is divided into a number of layers (levels), each built on top of lower layers. The bottom layer (layer 0), is the hardware; the highest (layer N) is the user interface.
- With modularity, layers are selected such that each uses functions (operations) and services of only lower-level layers

8/3/2023 21CSC202J Operating Systems

UNIT 1 95

[Page 96]

Microkernel System Structure

- Moves as much from the kernel into user space

- Mach example of microkernel
 - Mac OS X kernel (Darwin) partly based on Mach
- Communication takes place between user modules using message passing
- Benefits:
 - Easier to extend a microkernel
 - Easier to port the operating system to new architectures
 - More reliable (less code is running in kernel mode)
 - More secure
- Detriments:
 - Performance overhead of user space to kernel space communication

8/3/2023 21CSC202J Operating Systems
UNIT 1 96

[Page 97]

Microkernel System Structure

Application

Program

File

System

Device

Driver

Interprocess

Communication

memory

management

CPU

scheduling

messagesmessages

microkernel

hardware

user

mode

kernel

mode

8/3/2023 21CSC202J Operating Systems
UNIT 1 97

[Page 98]

Modules

- Many modern operating systems implement loadable kernel modules
 - Uses object-oriented approach
 - Each core component is separate

- Each talks to the others over known interfaces
- Each is loadable as needed within the kernel
- Overall, similar to layers but with more flexible
- Linux, Solaris, etc

8/3/2023 21CSC202J Operating Systems

UNIT 1 98

[Page 99]

Solaris Modular Approach

8/3/2023 21CSC202J Operating Systems

UNIT 1 99

[Page 100]

Hybrid Systems

- Most modern operating systems are actually not one pure model
- Hybrid combines multiple approaches to address performance, security, usability needs
- Linux and Solaris kernels in kernel address space, so monolithic, plus modular for dynamic loading of functionality
- Windows mostly monolithic, plus microkernel for different subsystem personalities
- Apple Mac OS X hybrid, layered, Aqua UI plus Cocoa programming environment
- Below is kernel consisting of Mach microkernel and BSD Unix parts, plus I/O kit and dynamically loadable modules (called kernel extensions)

8/3/2023 21CSC202J Operating Systems

UNIT 1 100

[Page 101]

Mac OS X Structure

graphical user interface Aqua

application environments and services

kernel environment

Java Cocoa Quicktime BSD

Mach

I/O kit kernel extensions

BSD

8/3/2023 21CSC202J Operating Systems

UNIT 1 101

[Page 102]

iOS

- Apple mobile OS for iPhone, iPad
- Structured on Mac OS X, added

functionality

- Does not run OS X applications natively

- Also runs on different CPU

architecture (ARM vs. Intel)

- Cocoa Touch Objective-C API for

developing apps

- Media services layer for graphics,

audio, video

- Core services provides cloud

computing, databases

- Core operating system, based on Mac

OS X kernel

8/3/2023 21CSC202J Operating Systems

UNIT 1 102

[Page 103]

Android

- Developed by Open Handset Alliance (mostly Google)

- Open Source

- Similar stack to IOS

- Based on Linux kernel but modified

- Provides process, memory, device-driver management

- Adds power management

- Runtime environment includes core set of libraries and

Dalvik virtual machine

- Apps developed in Java plus Android API

- Java class files compiled to Java bytecode then

translated to executable then runs in Dalvik VM

- Libraries include frameworks for web browser (webkit),

database (SQLite), multimedia, smaller libc

8/3/2023 21CSC202J Operating Systems

UNIT 1 103

[Page 104]

Android Architecture

Applications

Application Framework

Android runtime

Core Libraries

Dalvik

virtual machine

Libraries

Linux kernel

SQLite OpenGL

surface

manager

webkit libc

media

framework

8/3/2023 21CSC202J Operating Systems

UNIT 1 104

[Page 105]

Operating-System Debugging

- Debugging is finding and fixing errors, or bugs
- OS generate log files containing error information
- Failure of an application can generate core dump file capturing memory of the process
- Operating system failure can generate crash dump file containing kernel memory
- Beyond crashes, performance tuning can optimize system performance
 - Sometimes using trace listings of activities, recorded for analysis
 - Profiling is periodic sampling of instruction pointer to look for statistical trends

Kernighan's Law: "Debugging is twice as hard as writing the code in the first place. Therefore, if you write the code as cleverly as possible, you are, by definition, not smart enough to debug it."

8/3/2023 21CSC202J Operating Systems

UNIT 1 105

[Page 106]

Performance Tuning

- Improve performance by removing bottlenecks
- OS must provide means of computing and displaying measures of system behavior
- For example, "top" program or Windows Task Manager

Task Manager

8/3/2023 21CSC202J Operating Systems

UNIT 1 106

[Page 107]

DTrace

DTrace tool in Solaris,

FreeBSD, Mac OS X

allows live instrumentation on production systems

Probes fire when code is
executed within a provider,
capturing state data and
sending it to consumers of
those probes

Example of following

XEventsQueued system call
move from libc library to
kernel and back

8/3/2023 21CSC202J Operating Systems

UNIT 1 107

[Page 108]

Dtrace (Cont.)

DTrace code to record amount
of time each process with

UserID 101 is in running mode
(on CPU) in nanoseconds

8/3/2023 21CSC202J Operating Systems

UNIT 1 108

[Page 109]

Operating System Generation

n Operating systems are designed to run on any of a class of
machines; the system must be configured for each specific computer
site

n SYSGEN program obtains information concerning the specific
configuration of the hardware system

l Used to build system-specific compiled kernel or system-tuned

l Can generate more efficient code than one general kernel

8/3/2023 21CSC202J Operating Systems

UNIT 1 109

[Page 110]

System Boot

- When power initialized on system, execution starts at a fixed memory location
 - Firmware ROM used to hold initial boot code
- Operating system must be made available to hardware so hardware can start it
 - Small piece of code - bootstrap loader, stored in ROM or EEPROM
locates the kernel, loads it into memory, and starts it
 - Sometimes two-step process where boot block at fixed location loaded by
ROM code, which loads bootstrap loader from disk
- Common bootstrap loader, GRUB, allows selection of kernel from multiple
disks, versions, kernel options
- 110

- Kernel loads and system is then running

8/3/2023 21CSC202J Operating Systems

UNIT 1 110

===== SOURCE_FILE: sem 3/Operating Systems/Notes/Unit 3.url =====

CATEGORY: Notes

STATUS: url_reference_only

URL: https://docs.google.com/presentation/d/1yhwbr_TyOnhN0_a7DVM66HkyokiGKSMq/edit?usp=drive_link&ouid=104856686103320873965&rtpof=true&sd=true

===== SOURCE_FILE: sem 3/Operating Systems/Notes/Unit 4.url =====

CATEGORY: Notes

STATUS: url_reference_only

URL: https://docs.google.com/presentation/d/1yBWMnmc2wUY_D8FZmxylgV7nAQWx51VC/edit?usp=drive_link&ouid=104856686103320873965&rtpof=true&sd=true

===== SOURCE_FILE: sem 3/Operating Systems/PYQs/CT Papers.url =====

CATEGORY: PYQs

STATUS: url_reference_only

URL: https://drive.google.com/drive/folders/1J62hkGVN_URlqlrh4op0Fri6yzHqugkx?usp=drive_link

===== SOURCE_FILE: sem 3/Operating Systems/PYQs/Important Topics.url =====

CATEGORY: PYQs

STATUS: url_reference_only

URL: https://docs.google.com/document/d/1RXAHwTC7dWe5apNJAJEVPNY_y35UhhR0NWBlal5_e_A/edit?usp=sharing